

A PROJECT REPORT ON

DIGITAL DETECTIVE

File Forensics & Investigation Tool

"Analyze. Investigate. Understand."

Submitted in partial fulfilment of the requirements for the award of the degree of

[Bachelor of Technology in Computer Science & Engineering

Artificial Intelligence and Machine learning]

Submitted by

[Mayank Chaudhary]

[25BAI10888]

[DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING]

[VIT BHOPAL]

[Sehore]

Abstract

Digital Detective is a desktop file-forensics and investigation utility built entirely in Core Java (Swing and NIO) that allows an investigator to scan any folder on a local machine and obtain a complete forensic-style overview of its contents. The application recursively walks the selected directory using Java NIO's `Files.walkFileTree` API, computes a SHA-256 cryptographic hash for every file, and extracts key metadata such as size, creation and modification timestamps, and hidden/read-only attributes.

The collected data is analysed by a set of pure, stateless analyzer classes that detect duplicate files using an efficient size-then-hash grouping strategy, flag potentially suspicious files through a transparent rule-based scoring system (covering double extensions, executable files, suspicious keywords, and other heuristics), build a chronological timeline of file activity, and compute storage statistics rendered as hand-drawn Java2D pie and bar charts. An overall File Health Score summarises the investigation in a single explainable 0–100 value.

All processing happens strictly on the local machine — the application makes no network calls, uses no external database, and never modifies, deletes, or executes any scanned file. Results can be explored interactively through a dark, cybersecurity-themed Swing dashboard, or exported as a self-contained, professionally styled HTML report and a CSV inventory for further analysis in spreadsheet software. The project demonstrates practical application of object-oriented design, multithreading with `ExecutorService` and `SwingWorker`, Java collections, cryptographic hashing, and clean separation of concerns across model, scanner, analyzer, report, and UI layers, and is validated by a suite of JUnit 5 unit and integration tests.

Keywords: Digital Forensics, Java Swing, SHA-256 Hashing, Duplicate File Detection, Multithreading, File System Analysis, Heuristic Risk Scoring.

Table of Contents

Certificate	Error! Bookmark not defined.
Acknowledgement	Error! Bookmark not defined.
Abstract	2
Table of Contents	3
1. Introduction.....	5
1.1 Problem Statement	5
1.2 Objectives	5
1.3 Scope.....	6
1.4 Educational Disclaimer	6
2. Existing System and Motivation	6
3. System Requirements	7
3.1 Hardware Requirements	7
3.2 Software Requirements	7
4. System Analysis and Design.....	7
4.1 System Architecture	7
4.2 Use Case Diagram (Description)	8
4.3 Class Diagram (Description)	8
4.4 Data Flow Diagram	8
4.5 Sequence of a Full Scan	9
5. Technology Stack.....	9
<i>Why Swing instead of JavaFX?</i>	9
6. Module Description and Implementation	10
6.1 File Scanning Module	10
6.2 Metadata Extraction and Hashing Module	10
6.3 Duplicate Detection Algorithm	10
6.4 Suspicious File Analyzer	10
6.5 Timeline and Storage Analytics Modules	11
6.6 Forensic Score Calculator	11
6.7 Report Generation Module.....	12
6.8 Investigation History Module	12
6.9 Graphical User Interface Module.....	12
7. Key Features	12
8. Testing	13

8.1 Testing Strategy	14
8.2 Manual / GUI Testing	14
9. Results and Discussion	14
9.1 Sample Screens to Include	14
10. Limitations	15
11. Future Enhancements	15
12. Conclusion	15
13. References / Bibliography.....	16

1. Introduction

In an age where digital devices store an ever-growing volume of personal and professional data, the ability to quickly understand what is on a disk — what files exist, which ones are duplicated, which ones look out of place, and how storage is being consumed — is a valuable skill for students of cybersecurity, digital forensics, and system administration alike. Professional forensic suites exist, but they are often expensive, closed-source, or require installation of heavyweight third-party components, making them unsuitable for a classroom or a personal learning project.

Digital Detective was built to fill that gap: a fully self-contained, offline, Core Java desktop application that performs a forensic-style scan of any folder and presents the investigator with an organised, explainable, and visually engaging analysis — without requiring a database, an internet connection, or any paid software.

1.1 Problem Statement

Manually inspecting a large folder tree to find duplicate files, identify potentially disguised or suspicious files (such as an executable renamed to look like a document), understand how storage space is being used, and reconstruct a timeline of file activity is tedious, error-prone, and impractical to do by hand for anything beyond a handful of files. There is a need for a lightweight, transparent, and easy-to-run desktop tool that automates this analysis while clearly explaining its reasoning, rather than presenting an unexplained “black box” verdict.

1.2 Objectives

- To recursively scan a user-selected folder and build a complete inventory of every file, including its size, type, and timestamps.
- To compute a SHA-256 hash for every scanned file and use it to reliably detect byte-identical duplicate files.
- To design and implement a transparent, rule-based heuristic engine that flags potentially suspicious files (e.g., double extensions, executable files, suspicious keywords) with an explainable risk score — without ever claiming certainty of malicious intent.
- To visualise storage usage and file-type distribution using hand-drawn charts, and to present a chronological timeline of file activity.
- To keep the entire scan responsive by performing file I/O and hashing on background threads, never freezing the graphical user interface.
- To generate a professional, self-contained HTML forensic report and a CSV inventory export for further analysis, entirely offline.

- To demonstrate sound object-oriented design, Java NIO, multithreading, and unit testing practices suitable for academic evaluation.

1.3 Scope

The application operates entirely on the local file system of the machine it runs on. It analyses regular files and folders reachable from a chosen root directory and reports on their metadata and content hash. It does not inspect the byte-level content of files for embedded malicious code (it is not an antivirus), does not connect to any network or cloud service, does not maintain a persistent database, and does not automatically delete, modify, move, or execute any file it scans. Its suspicious-file detection and File Health Score are heuristic and educational in nature, intended to prompt manual review rather than serve as a definitive security verdict.

1.4 Educational Disclaimer

This tool is intended strictly for educational purposes and authorised forensic analysis of systems the user owns or has explicit permission to analyse. It is a college project demonstrating Java file handling, multithreading, hashing, and GUI development, and must never be used to analyse systems or files without proper authorisation.

2. Existing System and Motivation

Commercial and open-source digital forensics suites such as Autopsy, EnCase, and FTK provide powerful, feature-rich analysis of disk images, including file carving, registry analysis, and evidentiary chain-of-custody tracking. While extremely capable, these tools are generally designed for professional forensic examiners, require substantial installation and configuration, and are often overkill for a student wanting to understand the underlying concepts of file-system scanning, hashing, and duplicate/anomaly detection.

At the other end of the spectrum, simple duplicate-file finder utilities exist that only compare file sizes or names, which can produce false positives and provide no context about suspicious naming patterns or an overall risk picture. Digital Detective is positioned between these two extremes: it is simple enough to build, run, and explain within the scope of a college project, yet demonstrates genuine forensic techniques — cryptographic hashing, size-then-hash duplicate grouping, and transparent rule-based risk scoring — that mirror real-world practice on a much smaller and fully auditable scale.

The motivation for choosing Core Java Swing over a web-based or JavaFX-based interface was explicitly to keep the project runnable with zero external setup: it requires nothing beyond a standard JDK installation, has no third-party runtime dependencies, needs no internet access to run, and starts immediately with a single Maven command or a plain `javac/java` invocation.

3. System Requirements

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core 1.6 GHz or higher (multi-core recommended for faster parallel hashing)
RAM	4 GB minimum (8 GB recommended for scanning very large folder trees)
Storage	100 MB free disk space for the application and generated reports
Display	1366 × 768 resolution or higher

3.2 Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS, or Linux (cross-platform via JVM)
Java Development Kit	JDK 17 or newer (developed and tested on JDK 17 and JDK 21)
Build Tool	Maven 3.6+ (optional — plain javac/java also supported)
IDE (recommended)	Visual Studio Code with the Extension Pack for Java, or IntelliJ IDEA / Eclipse
Testing Framework	JUnit 5 (Jupiter) 5.10.2, used only for the test suite
GUI Toolkit	Java Swing (bundled with the JDK — no external download required)

4. System Analysis and Design

4.1 System Architecture

Digital Detective follows a simple layered architecture deliberately similar to MVC. Model classes carry data, scanner classes perform the raw file-system work, analyzer classes derive insights from the collected data, report classes render output files, and the ui package — the only layer that imports Swing — presents everything to the user. This separation means every package except ui is plain Java and could, in principle, be reused behind a different interface (a command-line tool or a web front end) without modification, a direct application of the Single Responsibility Principle.

Layer / Package	Responsibility
model	Plain data classes — FileRecord, FileType, Severity, SuspiciousFinding, and Investigation (the aggregate root holding an entire scan's results).

Layer / Package	Responsibility
scanner	Everything that touches the real file system — FileScanner (multithreaded NIO walk), MetadataExtractor (per-file metadata), HashCalculator (streaming SHA-256).
analyzer	Pure, stateless logic operating on collected FileRecord lists — duplicate grouping, suspicious scoring, timeline building, storage statistics, and health-score calculation, orchestrated by InvestigationAnalyzer.
report	Converts an Investigation into output — HtmlReportGenerator, CsvExporter, and InvestigationHistoryManager (local history, no database).
ui	All Swing screens — MainFrame (toolbar + sidebar + CardLayout shell), one JPanel per dashboard section, and shared Theme / table-model utilities.
util	Small stateless helpers — Constants, FileUtils, DateUtils, and DemoDataGenerator for the built-in Demo Mode.

4.2 Use Case Diagram (Description)

The primary actor is the Investigator, who can: select a folder and start a scan, load the built-in Demo Mode dataset, browse/search/filter the scanned files, review duplicate file groups, review suspicious findings, search by SHA-256 hash, view the activity timeline, view storage analytics, generate an HTML/CSV/summary report, add investigation notes, review investigation history, and stop an in-progress scan. The “Select folder & scan” use case includes the multithreaded scan-and-hash sub-process, and “Generate report” includes embedding the investigator's notes into the output.

4.3 Class Diagram (Description)

The central class is Investigation, which aggregates a list of FileRecord objects (one per scanned file, carrying name, path, type, size, timestamps, SHA-256 hash, hidden/read-only flags, and an optional SuspiciousFinding), a list of duplicate groups, a list of SuspiciousFinding objects, a file-type distribution map, and the overall forensic score. FileScanner depends on MetadataExtractor and HashCalculator to populate FileRecord instances and produces an Investigation. DuplicateAnalyzer and SuspiciousAnalyzer are stateless utility classes that read FileRecord lists and produce derived results; SuspiciousFinding in turn carries a Severity enum (LOW/MEDIUM/HIGH).

4.4 Data Flow Diagram

Data flows in one direction from the file system through the scanning layer into the in-memory Investigation model, then out to the analyzers (which write derived fields back into the same Investigation), and finally out to either the Swing UI panels or the report/history writers on local disk. Crucially, every arrow in the data flow terminates at either the Swing UI or the local disk — there is no network call anywhere in the pipeline, which is a deliberate, auditable design choice underpinning the tool's local-first, privacy-respecting design.

4.5 Sequence of a Full Scan

When the user selects a folder and clicks Start Scan, MainFrame launches a SwingWorker so the Event Dispatch Thread is never blocked. The worker calls `FileScanner.scan()`, which walks the directory tree with `Files.walkFileTree()` and, for every file discovered, submits a task to a fixed `ExecutorService` thread pool. Each pooled task calls `MetadataExtractor.extract()` and `HashCalculator.sha256()` and adds the resulting `FileRecord` to the Investigation, publishing progress back to the GUI thread. Once the walk completes and all pooled tasks finish, `InvestigationAnalyzer.runAll()` computes duplicates, suspicious findings, storage statistics, and the health score; MainFrame then refreshes every panel and `InvestigationHistoryManager` saves a lightweight history entry with an HTML snapshot.

5. Technology Stack

A deliberate design decision of this project was to rely exclusively on the Java Standard Library for the application itself, with JUnit added only as a test-scope dependency. This keeps the project dependency-free, easy to build offline, and simple to explain and defend in a viva.

Technology	Used For
Java 17+	Core application language (developed and tested on JDK 17 and JDK 21)
Java Swing	Desktop GUI — chosen over JavaFX because it ships inside every standard JDK with zero extra setup
Java NIO (<code>java.nio.file</code>)	Recursive directory walking (<code>Files.walkFileTree</code>) and file metadata access
<code>java.security.MessageDigest</code>	Streaming SHA-256 hashing of every scanned file
<code>java.util.concurrent</code>	<code>ExecutorService</code> thread pool, <code>SwingWorker</code> , <code>AtomicBoolean</code> / <code>AtomicInteger</code> for responsive multithreaded scanning
<code>java.time</code>	Timestamp handling for created/modified/accessed metadata and the timeline view
Java2D (<code>Graphics2D</code>)	Hand-drawn pie and bar charts for storage analytics — no external charting library required
JUnit 5 (Jupiter)	Unit and integration testing of core logic
Maven	Build and dependency management

Why Swing instead of JavaFX?

JavaFX was considered but rejected because, as of Java 11 onward, it is no longer bundled with the JDK and requires either a separate SDK download with module-path VM arguments, or a Maven plugin that pulls platform-specific native binaries — adding friction on machines with limited internet access. Java

Swing ships inside every standard JDK, so the project can be compiled and run immediately, everywhere, with zero extra setup, while still being styled to a professional, cohesive dark theme.

6. Module Description and Implementation

6.1 File Scanning Module

The scanner package is responsible for all real file-system access. FileScanner performs a recursive walk of the selected root directory using `Files.walkFileTree()`, which is inherently single-threaded, but dispatches the expensive per-file work — metadata extraction and hashing — to a fixed `ExecutorService` thread pool sized to `Runtime.availableProcessors()`. This keeps directory traversal fast while making full use of available CPU cores for hashing. The entire scan runs inside a `SwingWorker`'s background thread, so the Event Dispatch Thread (EDT) never touches the file system and the GUI remains fully responsive, with progress published back to a live progress dialog that includes a working Stop Scan button.

The scanner is deliberately resilient: `visitFileFailed` and `postVisitDirectory` callbacks, together with a catch-all around per-file processing, ensure that permission-denied folders, broken symbolic links, and files deleted mid-scan are recorded as access errors and skipped rather than crashing the scan.

6.2 Metadata Extraction and Hashing Module

`MetadataExtractor` reads each file's size, creation/modification timestamps, and hidden/read-only attributes via NIO attribute views, falling back to POSIX-style checks on macOS/Linux where Windows-specific DOS attributes are unavailable. `HashCalculator` computes a SHA-256 digest by opening the file as a streaming `InputStream` and updating the digest in fixed 8 KB chunks — the file is never loaded fully into memory, allowing even multi-gigabyte files to be hashed safely with constant memory usage.

6.3 Duplicate Detection Algorithm

`DuplicateAnalyzer.findDuplicateGroups()` implements a two-stage strategy. Stage one groups all files by size using a `HashMap<Long, List<FileRecord>>`, since two files can only be byte-identical if they share the same size — this discards the overwhelming majority of non-duplicate candidates in a single $O(n)$ pass. Stage two takes every size bucket containing two or more files and further groups them by their already-computed SHA-256 hash; only hash buckets that still contain two or more files are reported as genuine duplicate groups. Because every file's hash was already computed once during the initial scan (and is reused by the Hash Explorer and report exports), `DuplicateAnalyzer` never re-hashes anything. This design achieves $O(n)$ average-case complexity, compared to $O(n^2)$ for a naive pairwise comparison of every file's hash against every other file's.

6.4 Suspicious File Analyzer

`SuspiciousAnalyzer.evaluate()` implements a fully transparent, rule-based scoring system — deliberately avoiding any black-box machine-learning model so that every score can be explained and defended. Each matching signal adds a fixed number of points to a 0–100 risk score:

Signal	Points	Rationale
Decoy double extension (e.g. invoice.pdf.exe)	+35	Classic social-engineering disguise pattern
Executable file extension	+25	Executables inherently carry higher risk than static documents
Suspicious keyword in file name (keygen, crack, payload, ...)	up to +40	Common naming pattern for unwanted software
Hidden and executable	+15	Hidden executables are unusual for normal user files
Executable created within the last 24 hours	+10	Freshly dropped executables warrant closer review
Executable in a commonly-abused folder (Temp/Downloads/Cache)	+10	Matches common malware staging locations
Hidden + read-only + executable together	+5	Unusual combination for a legitimate file

The total score is capped at 100. A file is only surfaced as a finding once its score reaches at least 20 (the `MIN_SCORE_TO_FLAG` threshold), avoiding noisy low-value results. `Severity.fromScore()` then buckets the score into LOW (20–34), MEDIUM (35–69), or HIGH (70–100). Notably, the double-extension check is not a naive “more than one dot” test — which would incorrectly flag legitimate files such as `archive.tar.gz` or `app.min.js` — but instead fires only when the final extension is itself executable and the extension immediately before it is a common, innocent-looking one (`pdf`, `jpg`, `docx`, `mp3`, and similar), precisely matching the `name.decoy.exe` disguise pattern. Every finding also carries a recommendation string that always reads “review manually” and never asserts that a file is definitely malicious.

6.5 Timeline and Storage Analytics Modules

`TimelineAnalyzer` builds a chronological view of created, modified, and accessed file events with quick range filters (Today, Last 24 Hours, Last 7 Days, Last 30 Days). `StorageAnalyzer` computes aggregate storage statistics and a file-type distribution across twelve categories (Documents, Images, Videos, Audio, Archives, Executables, Source Code, Spreadsheets, Presentations, Text, Database, Unknown), which are rendered as hand-drawn Java2D pie and bar charts alongside a Top 10 Largest Files list — avoiding any dependency on an external charting library.

6.6 Forensic Score Calculator

ForensicScoreCalculator combines the number of suspicious files, duplicate files, hidden files, empty files, and the executable-to-total-file ratio into a single transparent 0–100 File Health Score, explicitly documented as a heuristic summary rather than a malware or security verdict.

6.7 Report Generation Module

HtmlReportGenerator builds one self-contained HTML document (inline CSS, no external assets, dark theme matching the desktop UI) covering the header and scan metadata, summary statistic cards, investigator notes, file-type distribution, the top-largest-files table, duplicate groups, suspicious findings with severity colour-coding, hidden/empty files, a recent timeline, and recommendations. Every dynamic string is HTML-escaped to safely handle unusual file names, and very large sections are capped with a note pointing to the CSV export so the report stays a manageable size even for large investigations. CsvExporter writes the complete file inventory as RFC 4180-style CSV, correctly quoting and escaping values containing commas, quotes, or newlines, so it opens cleanly in Excel or Google Sheets.

6.8 Investigation History Module

Per the project brief, no database is used. InvestigationHistoryManager instead appends one row per completed scan to a local CSV file (investigation_history.csv, stored under the user's home directory) and saves a full HTML report snapshot alongside it, allowing the History screen to reopen any previous investigation's report instantly without re-scanning.

6.9 Graphical User Interface Module

The ui package is the only layer that imports Swing. WelcomeScreen presents the application's motto and disclaimer; MainFrame is the shell, combining a toolbar, a sidebar, and a CardLayout that switches between nine dashboard sections — Overview, Files, Duplicates, Suspicious, Timeline, Storage, Hash Explorer, Reports, and History — each implemented as its own JPanel with a refresh(Investigation) method. A shared Theme class centralises the dark colour palette and typography, and reusable UiUtils / FileTableModel components provide a single, consistently styled file table used across six different screens, ensuring visual consistency throughout the application.

7. Key Features

- Recursive folder scanning using Java NIO, with graceful handling of permission errors, broken symlinks, and files deleted mid-scan.
- Multithreaded scanning — a fixed ExecutorService thread pool hashes files in parallel while the GUI stays fully responsive, with a live progress dialog and a working Stop Scan button.
- SHA-256 hashing of every file, searchable from a dedicated Hash Explorer screen.
- Efficient duplicate file detection using two-stage size-then-hash grouping.

- Transparent, rule-based suspicious file analyzer producing an explainable 0–100 risk score and LOW/MEDIUM/HIGH severity.
- File type detection across twelve categories.
- Timeline view of file activity with quick range filters.
- Storage analytics with hand-drawn Java2D pie and bar charts, plus a Top 10 Largest Files list.
- Hidden-file and empty-file detection.
- Searchable, filterable File Explorer table (by name, extension, path, hash, or type).
- A transparent, explainable File Health Score summarising the whole investigation.
- Professional, self-contained HTML report generation and CSV inventory export, entirely offline.
- Lightweight local investigation history (CSV + saved HTML snapshots), with no database required.
- Investigation notes that are embedded directly into the generated report.
- A built-in Demo Mode that loads a realistic synthetic investigation for instant exploration or presentation.
- A polished, dark, cybersecurity-inspired Swing user interface.

8. Testing

The project includes a dedicated JUnit 5 test suite under `src/test/java`, comprising seven test classes covering both unit-level logic and integration-style scenarios across roughly 490 lines of test code. Testing focuses on the pure, stateless analyzer and utility logic, which is the easiest and most valuable code to test exhaustively, while GUI classes are exercised manually through the application's Demo Mode.

Test Class	Focus Area
FileScannerTest	Recursive directory walking, file discovery, and resilience to scan errors
HashCalculatorTest	Correctness of streaming SHA-256 hashing against known digests
FileUtilsTest	Size formatting, extension parsing, and the double-extension disguise detector
FileTypeTest	Correct classification of file extensions into the twelve supported categories
DuplicateAnalyzerTest	Correctness of the size-then-hash duplicate grouping algorithm
SuspiciousAnalyzerTest	Correctness of each individual risk-scoring rule and the resulting severity bucket
InvestigationTest	Aggregate behaviour of the Investigation model (counts, totals, derived fields)

8.1 Testing Strategy

- Unit tests validate individual algorithms in isolation — for example, confirming that `archive.tar.gz` is never flagged as a double-extension disguise while `invoice.pdf.exe` correctly is.
- Integration-style tests (e.g., `FileScannerTest`) exercise the scanner against real temporary directories created and torn down for each test run, verifying that scanning genuinely walks the file system and produces correct `FileRecord` data.
- Edge cases explicitly covered include empty files, hidden files, zero-byte duplicates, files with no extension, and folders containing files that become unreadable mid-scan.
- The full suite can be run with a single command: `mvn test`.

8.2 Manual / GUI Testing

Since Swing components are not covered by the automated suite, the application's nine dashboard screens were manually verified end-to-end using the built-in Demo Mode, which loads a realistic synthetic dataset covering duplicates, suspicious files across all severities, hidden and empty files, and a spread of file types — allowing every screen and chart to be exercised without needing to scan a real folder.

9. Results and Discussion

The completed application successfully scans folder trees of varying sizes, correctly identifies duplicate files sharing an identical SHA-256 hash, and flags disguised or suspicious files with an explainable score and severity. Because the directory walk and hashing are decoupled across a thread pool, the graphical interface remains responsive throughout a scan, with live progress feedback rather than an unresponsive “frozen” window.

The generated HTML report and CSV export were verified to open correctly in a standard web browser and in spreadsheet software respectively, confirming that the offline, database-free reporting pipeline works as intended. [Insert screenshots of the Welcome Screen, Dashboard/Overview, File Explorer, Duplicates, Suspicious Findings, Storage Analytics, and the generated HTML report here, with a short caption under each.]

9.1 Sample Screens to Include

- Welcome Screen — motto and disclaimer.
- Overview / Dashboard — files scanned, total size, duplicates, suspicious files, and the File Health Score.
- File Explorer — the searchable, filterable file table.
- Duplicates — a duplicate group with its shared SHA-256 hash and wasted-storage calculation.
- Suspicious Findings — an example such as `invoice.pdf.exe` with its score breakdown.

- Hash Explorer — searching by a copied SHA-256 hash.
- Timeline — a quick range filter such as “Last 7 Days”.
- Storage Analytics — the pie/bar charts and Top 10 Largest Files list.
- Reports — the generated HTML forensic report.

10. Limitations

- Heuristic, not a security product — the suspicious-file analyzer and File Health Score are transparent, rule-based heuristics for educational review; they do not inspect file contents for actual malicious code and must never be treated as a malware verdict.
- The scan progress bar is indeterminate rather than percentage-based, since computing an exact file count ahead of time would require a full extra directory pass.
- No database is used — investigation history is stored as local CSV and HTML snapshots rather than a queryable database, which limits history search capability but was an intentional design choice per the project brief.
- Windows-specific metadata such as the DOS read-only attribute falls back to POSIX-style checks on macOS/Linux, so “Read Only” detection is best-effort cross-platform.
- Very large folder trees (millions of files) will use proportionally more memory, since the current implementation keeps the full file inventory in memory for interactivity.

11. Future Enhancements

- An optional two-pass scan to display an exact percentage-based progress bar instead of an indeterminate one.
- Pluggable “magic byte” content sniffing to detect files whose actual content does not match their claimed extension (e.g., a renamed executable with a .pdf extension).
- Exportable, paginated history search backed by JSON-based history storage.
- A pluggable rules engine so suspicious-file heuristics can be tuned or extended without recompiling the application.
- An optional JavaFX build variant behind a Maven profile for users who have the JavaFX SDK already configured.

12. Conclusion

Digital Detective successfully demonstrates that a genuinely useful, forensic-style file-analysis tool can be built entirely on the Java Standard Library, without any third-party runtime dependencies, external database, or network connection. The project applies core computer-science and software-engineering concepts — recursive tree traversal, cryptographic hashing, efficient hash-map-based duplicate detection,

transparent rule-based heuristic scoring, concurrent programming with thread pools and `SwingWorker`, and clean layered object-oriented design — to a practical, well-motivated problem.

The resulting application is easy to build and run on any machine with a standard JDK, remains responsive during large scans thanks to its multithreaded architecture, and produces professional, self-contained reports suitable for sharing or archival. Beyond its immediate functionality, the project offers a solid foundation for further exploration of digital forensics concepts and demonstrates, in an auditable and explainable way, how heuristic risk-scoring systems can be designed responsibly — always recommending manual review rather than asserting certainty.

13. References / Bibliography

1. Oracle Corporation, “Java Platform, Standard Edition Documentation”, docs.oracle.com/en/java/javase/.
2. Oracle Corporation, “`java.nio.file` Package Summary — `Files.walkFileTree`”, Java SE API Documentation.
3. Oracle Corporation, “`java.security.MessageDigest` Class”, Java SE API Documentation.
4. Oracle Corporation, “`java.util.concurrent` Package — `ExecutorService`”, Java SE API Documentation.
5. Oracle Corporation, “`SwingWorker` Class — Lesson: Concurrency in Swing”, The Java Tutorials.
6. National Institute of Standards and Technology (NIST), “Secure Hash Standard (SHS)”, FIPS PUB 180-4.
7. The JUnit Team, “JUnit 5 User Guide”, junit.org/junit5/docs/current/user-guide/.
8. The Apache Software Foundation, “Maven – Introduction to the POM”, maven.apache.org.
9. Project source code, `README.md`, `ARCHITECTURE.md`, and `VIVA.md` — Digital Detective repository.